

Mattermost Cross Guard Threat Model

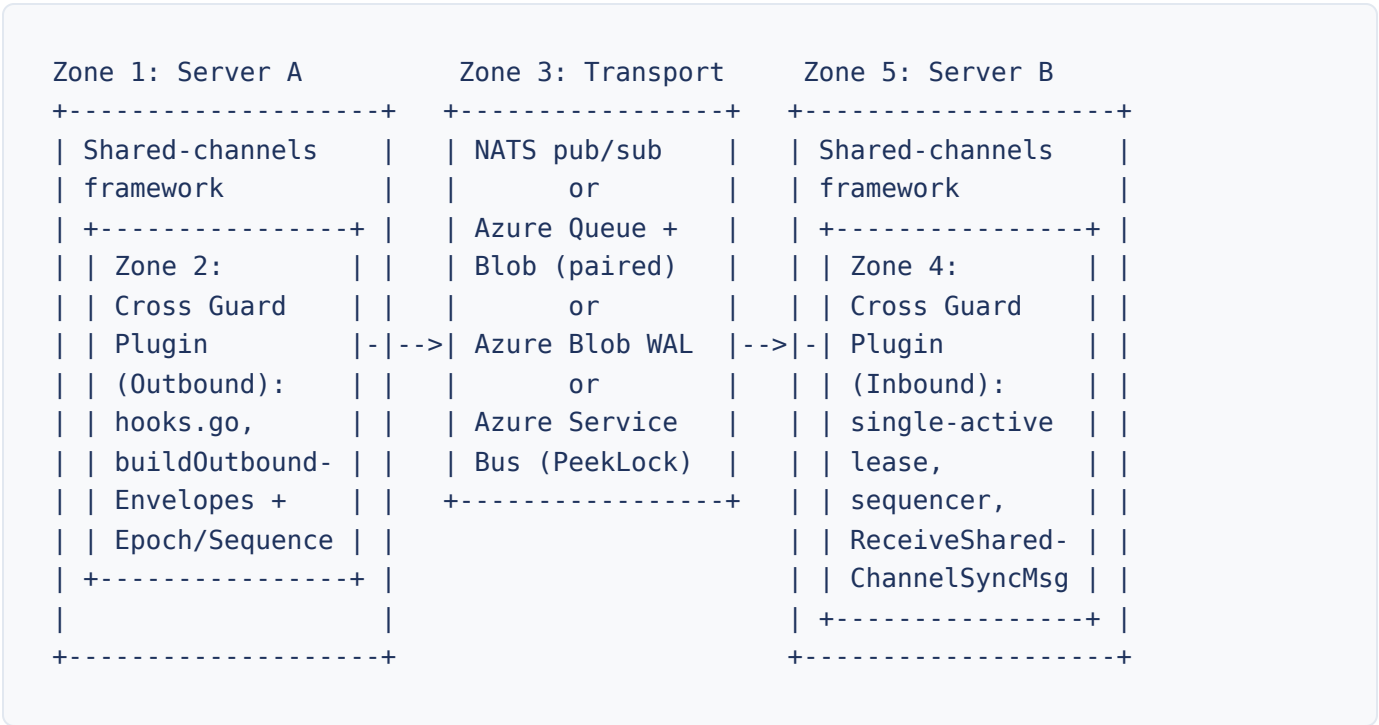
Security analysis of Cross Guard's cross-domain relay architecture

Introduction

Cross Guard operates as a Cross Domain Solution (CDS), bridging two or more Mattermost instances across security domain boundaries using one of four pluggable transport providers: NATS, Azure Queue Storage, Azure Blob Storage (batched WAL), or Azure Service Bus. The plugin integrates with Mattermost's shared-channels framework, marshalling the framework's sync events into an XML (or JSON, per `message_format`) envelope and handing them to the configured transport. This page documents the trust boundaries, the data that crosses them, and the threats and mitigations at each layer.

Intended audience: security reviewers, accreditors, and administrators evaluating Cross Guard for deployment in classified or sensitive environments.

System Architecture and Trust Zones



Zone	Description	Trust Level	Controlled By
Zone 1: Mattermost Server A	Source Mattermost instance: users, teams, channels, posts. The shared-channels framework owns post-state events and drives outbound sync via its hooks.	Trusted	Server A admin
Zone 2: Cross Guard Plugin (Outbound)	Plugin code running inside Server A. Implements <code>OnSharedChannelsSyncMsg</code> , <code>OnSharedChannelsAttachmentSyncMsg</code> , <code>OnSharedChannelsProfileImageSyncMsg</code> , and <code>OnSharedChannelsPing</code> in <code>server/hooks.go</code> . Projects upstream <code>mmModel</code> types into plugin-owned wire types (<code>server/wire/</code>), fans out one envelope per <code>Post</code> (<code>buildOutboundEnvelopes</code> in <code>server/transport.go</code>), stamps Epoch and Sequence (<code>publishToOutboundConn</code> in <code>server/connections.go</code>), and publishes via <code>QueueProvider.Publish</code> . File payloads ride a parallel channel through <code>QueueProvider.UploadFile</code> .	Trusted (same process as Mattermost)	Plugin code + Server A admin config
Zone 3: Transport Layer	One of four providers: a NATS server (core pub/sub plus JetStream Object Store for files), Azure Queue Storage (with paired Blob container for files), Azure Blob Storage (batched WAL plus deferred file blobs in the same container), or Azure Service Bus (PeekLock queue plus an independent Blob sidecar for files). The plugin trusts the transport for delivery only, not for content integrity.	Semi-trusted (transport security only)	Transport admin / Azure subscription owner (may be separate from MM admins)
Zone 4: Cross Guard Plugin (Inbound)	Plugin code running inside Server B. Exactly one node per cluster holds the active subscription via a KV TTL lease (<code>server/inbound_election.go</code>). Inbound bytes flow through <code>processInboundMessage</code> → <code>UnmarshalEnvelope</code> → <code>inboundSequencer.Admit</code> (dedup, reorder, gap audit; <code>server/inbound_sequencer.go</code>)	Trusted (same process as Mattermost)	Plugin code + Server B admin config

Zone	Description	Trust Level	Controlled By
	→ <code>rewriteChannelIDs</code> → <code>ReceiveSharedChannelSyncMsg</code> . File payloads ride <code>QueueProvider.WatchFiles</code> with the same single-active lease.		
Zone 5: Mattermost Server B	Destination Mattermost instance. The shared-channels framework applies received sync content to local state, stamping each remote user with a non-empty <code>RemoteId</code> so framework cursors do not re-emit it.	Trusted	Server B admin

Data Crossing the Domain Boundary

Two distinct delivery paths carry data across the trust boundary:

- **XML / JSON envelope path.** The `<CrossGuardEnvelope>` wrapper, carrying either a `SyncMsg` payload (`type="sync_msg"`) or a `TestID` (`type="test"`). The wire types live in `server/wire/` ; the structural contract is `schema/crossguard.xsd` . JSON and XML are field-for-field equivalent and chosen per outbound connection via the `message_format` setting; inbound auto-detects by the first non- whitespace byte (`<` for XML, `{` for JSON; a leading UTF-8 BOM on XML is stripped before detection). 22 byte-stable XML fixtures under `schema/examples/` are the canonical reference; `TestExampleFiles` in `server/examples_roundtrip_test.go` enforces byte equality and round-trip determinism.
- **File-upload path.** Raw attachment and profile-image bytes with HTTP-style `X-Crossguard-*` headers carrying the routing metadata (no XML envelope). Each transport maps the header set onto its native metadata mechanism (NATS object store headers, Azure Blob metadata, Azure Service Bus blob sidecar metadata). The full header set is documented in [File-Upload Headers](#).

Envelope Schema (`TransportEnvelope` in `server/transport.go`)

version (attribute)	Wire-format version, currently always <code>1</code> .
type (attribute)	One of <code>sync_msg</code> , <code>test</code> , <code>attachment</code> , <code>profile_image</code> . Only <code>sync_msg</code> and <code>test</code> ride this envelope; the other two are file-upload kinds (see above).

ConnName	Outbound connection name (URL-safe slug, max 96 chars). Used to scope per-channel sequence numbers and to dispatch audit events.
Timestamp	RFC 3339 UTC timestamp; defaults to marshal time when caller leaves it empty.
Epoch	Sender process generation (26-char Mattermost ID, generated at <code>OnActivate</code>). Lets the receiver correlate sequence numbers across restarts and resets the per-channel cursor on a new epoch.
Sequence	Per- (<code>ConnName</code> , <code>ChannelId</code>) monotonic counter. <code>sync_msg</code> only; absent on <code>test</code> envelopes.
TeamName	Source team name (URL-safe slug, max 96 chars). Used for inbound routing via the rewrite index and name lookup.
ChannelName	Source channel name (URL-safe slug, max 96 chars). Used for inbound name resolution.
SyncMsg	Plugin-owned <code>wire.SyncMsg</code> payload (present when <code>type</code> is <code>sync_msg</code>).
TestID	26-char ID echoed by the receiver into audit logs (present when <code>type</code> is <code>test</code>).

SyncMsg Payload Contents

The `syncMsg` payload is a plugin-owned wire-pruned subset of Mattermost's upstream `mmModel.SyncMsg` . Each envelope carries at most one `Post` : the sender fans out an upstream sync cycle into one envelope per `Post` plus an optional metadata envelope carrying non-post content (orphan reactions, memberships, statuses, acknowledgements, mention transforms). This one-post-per-envelope contract is the compliance-content-inspection blast-radius control: a content inspector that rejects one envelope does not lose other content as collateral damage. The full per-type field reference is in [SyncMsg Payload](#).

The compliance-relevant constraints, summarized:

- **Whitelisted typed Props.** `Post.Props` and `User.Props` are not pass-through maps. They are typed structs (`PostProps` , `UserProps`) with a fixed key list (15 keys on `PostProps` , 4 on `UserProps`). Any non-whitelisted upstream key is dropped at the boundary regardless of which plugin set it.
- **Dropped at source.** Receiver-overwritten and source-only fields (`Post.ReplyCount` , `Post.LastReplyAt` , `Post.Participants` , `Post.MessageSource` , `Post.PendingPostId` ,

`Post.Filenames` , `Post.Metadata` , plus credentials, MFA state, and notification preferences on `User`) never appear on the wire.

- **Username scrubbing.** Colon-tagged remote usernames (`alice.high`) are resolved to their pre-tag form before being placed in the wire `Username` field. The colon-bearing form, if needed downstream, is recovered from `UserProps.RemoteUsername` .
- **Status pruning.** Source-local `ActiveChannel` on presence updates is dropped at the boundary.
- **Bounded length fields.** Every string in the XSD carries a `maxLength` (post message body capped at 1 MiB; URL-safe slugs capped at 96; emails capped at 381); every list carries a concrete `maxOccurs` upper bound (Fileids=10; Users / Reactions / Statuses / etc.=1000). Marshalling does not produce an envelope that exceeds these caps.

File-Upload Headers (`attachment` , `profile_image`)

File payloads do not ride the XML / JSON envelope. They flow through

`QueueProvider.UploadFile` with headers carrying the routing metadata. The header set is documented in [File-Upload Headers](#); the compliance-relevant facts:

- **Opt-in per connection.** File transfer is disabled by default; each connection enables it via `file_transfer_enabled` .
- **Extension filtering.** Per-connection allow / deny extension policy via `file_filter_mode` and `file_filter_types` . Applied on both outbound and inbound; a file blocked on either side is dropped.
- **FileInfo wrapping.** Attachments carry an `X-File-Info` header containing a Base64-encoded JSON `mmModel.FileInfo` . Base64 wraps the JSON so it survives provider metadata-encoding rules; the receiver decodes it and hands off to the framework's attachment-upsert API.
- **Lane discipline.** Object keys follow stable patterns (`attachment/{postID}/{fileID}` , `profile_image/{userID}`); the receiver cross-checks the key prefix against the `X-Crossguard-Kind` header so a mistyped upload cannot escape its lane.
- **Size bound.** Mattermost's `FileSettings.MaxFileSize` (default 100 MB) bounds attachment size.

Threat Analysis (STRIDE)

Each threat is identified by a unique ID, categorized by STRIDE, and assessed for severity. Code references point to the specific source files where the behavior occurs.

Spoofing

T-SPOOF-1: Remote User Impersonation

Severity	MEDIUM
Description	The wire <code>User.Username</code> field is user-controlled at the source. An attacker controlling the remote Mattermost instance (or with transport publish access) can fabricate sync envelopes whose <code>User</code> records carry any username, email, or display-name they choose. The shared-channels framework upserts these as remote users on the local side, distinguished by a non-empty <code>RemoteId</code> ; they cannot silently merge into existing local accounts that have a different <code>RemoteId</code> (including the empty <code>RemoteId</code> on native users). However, a fabricated remote user is still visible in the channel and can be made to look very similar to a real local user (matching display name, similar username) if no operator is paying attention.
Code Path	<code>processInboundMessage</code> in <code>server/inbound.go</code> hands the unmarshalled <code>wire.SyncMsg</code> to <code>p.API.ReceiveSharedChannelSyncMsg</code> , which performs the upsert. The framework stamps every received user with the source <code>RemoteId</code> and refuses to write to existing local user IDs that belong to a different remote.
Mitigation	1. The framework's <code>RemoteId</code>-based isolation prevents the high-severity case (silent overwrite of a local user). Synced users are always distinguishable from native users by their non-empty <code>RemoteId</code> . 2. The webapp's <code>CrossguardUserPopover</code> component surfaces "Relayed from: alice (via high-side)" attribution on any synced user, derived from <code>UserProps.RemoteUsername</code> and the source <code>RemoteId</code> . Operators viewing a channel can always tell which conversation came from which domain. 3. Restrict transport publish access (NATS ACLs scoped to <code>crossguard.*</code> ; Azure RBAC scoped to specific queues / containers / namespaces; Service Bus SAS rules scoped to one queue) so only authorized Mattermost servers can put envelopes on the wire. Combine with network-level controls (IPsec, firewall rules, Azure Private Endpoints). 4. Audit synced remote users periodically by enumerating users with a non-empty <code>RemoteId</code> and comparing against the expected remote topology.
Residual Risk	A compromised but network-authorized remote can still create synced users with any (username, display name) combination they choose. The framework limits

impersonation to "looks similar to a local user"; it does not prevent it. End-to-end content authentication would close this gap and is not currently implemented.

T-SPOOF-2: Transport Message Injection (No Per-Message Authentication)

Severity

MEDIUM

Description

The wire envelope (`TransportEnvelope` in `server/transport.go`) contains no cryptographic signature, HMAC, or message authentication code. Any entity with publish access to the configured transport (NATS subject, Azure queue, Azure Blob container, or Azure Service Bus queue) can inject arbitrary envelopes in either JSON or XML that the inbound handler will admit, sequencer permitting.

Code Path

`processInboundMessage` in `server/inbound.go` calls `UnmarshalEnvelope` , which auto-detects the format and parses bytes with no signature verification. The envelope is then admitted by `inboundSequencer.Admit` (which catches duplicates and reorders, but does not authenticate content) before being handed to the framework.

Mitigation

1. TLS protects against passive eavesdropping and on-wire tampering between authorized clients and the transport. All four providers use TLS by default (NATS via the `tls_enabled` setting and minimum TLS 1.2; Azure providers via TLS-always endpoints).
2. Per-connection transport authentication (NATS token / credentials / mTLS; Azure shared key, SAS, or Service Principal; see Section 11) restricts who can connect.
3. Restrict transport network access to only authorized Mattermost servers using IPsec, firewall rules, network segmentation, or Azure Private Endpoints. Configure NATS ACLs or Azure RBAC so only authorized clients can publish to the specific resource Cross Guard uses.
4. The inbound sequencer catches replay attacks: an attacker who captures and re-emits a previously-seen envelope is rejected by the (`Epoch` , `ConnName` , `ChannelId` , `Sequence`) dedup. See T-TAMP-3 for the sequencer's role.

Residual Risk

A compromised but network-authorized client with valid transport credentials can still inject novel envelopes (not replays). No end-to-end content integrity exists; the framework on the receiving side trusts the parsed envelope. Combine network-level restrictions with the principle of least transport privilege per direction (separate subjects / queues / namespaces per relay direction) to limit blast radius.

T-SPOOF-AZ1: Azure Credential Compromise

Severity

HIGH

Description

An attacker who obtains the Azure Storage connection string or shared-key (Azure Queue or Azure Blob providers), the Service Bus SAS connection string (Azure Service Bus provider), or the Service Principal client secret (any of the three providers when `auth_mode` is `service-principal`) can send or read messages on the corresponding transport. Shared-key credentials provide full account access; a SAS connection string built from `RootManageSharedAccessKey` grants full namespace control, while a scoped SAS rule grants only the rights it was created with. A Service Principal credential grants exactly the Azure RBAC roles assigned to the SP at the configured scope. Mattermost does not encrypt plugin settings at rest: by default the connections JSON (containing the shared-key, SAS, or `client_secret` value) lives as plaintext in `config.json` on disk and in the Mattermost configuration database, where any operator with disk, backup, or DB access can read it. The Mattermost `GET /api/v4/config` endpoint also returns these secrets in cleartext to any System Admin.

Mitigation

- 1. Prefer Service Principal over shared-key / SAS.** An SP credential can be scoped via Azure RBAC to the minimum data-plane role required (e.g., `Storage Queue Data Contributor` on one storage account, `Azure Service Bus Data Sender` + `Receiver` on one queue) instead of granting account-wide or namespace-wide control, and rotation happens in Azure AD without touching plugin config. See the admin guide's [Azure Service Principal Authentication](#) section for the minimum RBAC roles and Phase 1 deployment guidance.
- 2. Keep the SP secret out of plugin config on disk.** Inject the entire connections JSON via Mattermost's plugin-settings env-var substitution (`MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` and the inbound counterpart). That env var can be sourced from a Kubernetes Secret, a CSI volume + init container, `systemd LoadCredential=`, or HashiCorp Vault. With this in place the `client_secret` never lands in `config.json` on disk, in the configuration database, or in a `GET /api/v4/config` response.
- 3. Inject the connections JSON via environment variables instead of saving it through the System Console.** Mattermost substitutes plugin settings from environment variables of the form `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` and `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_INBOUNDCONNECTIONS` at config-load time. The credential then lives only in the Mattermost process environment (and any orchestrator-managed secret store such as Kubernetes secrets, `systemd LoadCredential=`, or HashiCorp Vault that produced it), not in `config.json` on disk and not in the Mattermost configuration database. This is the recommended pattern for CDS deployments and is compatible with both legacy shared-key/SAS and Service Principal credentials.
- 4. Prefer scoped Service Bus SAS rules over the namespace-wide `RootManageSharedAccessKey`** when using connection-string mode, so an exposed credential grants only the minimum needed rights (Send / Listen / Manage scoped to one queue).

5. Rotate Azure Storage account keys, Service Bus SAS keys, and Service Principal client secrets on a regular schedule. SP rotation happens in Azure AD and is picked up at the next plugin construction (plugin reload or Mattermost restart); shared-key/SAS rotation requires updating the env-var-managed secret as well.
6. The Azure providers sanitize credential material out of error messages (account keys, SAS query params, and SP secret values via the shared `sanitizeAzureError()` path) before logging or returning to the admin UI, so a compromised log file does not leak the credential. The save-time SP probe applies the same sanitization to AAD errors.
7. Use Azure Private Endpoints to restrict network access to the storage account and Service Bus namespace. Monitor Azure Storage analytics and Service Bus diagnostic logs for anomalous access patterns. For SP, also monitor Azure AD sign-in logs for the SP's `appId` to detect unexpected token issuance.

Residual Risk

With env-var injection of the connections JSON, the credential is not on disk or in the Mattermost DB, but it is still readable by anyone who can read the Mattermost process environment (root on the host, container exec, `/proc/<pid>/environ`, container orchestrator logs). Without that indirection, the credential sits in plaintext in `config.json`, in the configuration database, and in any `GET /api/v4/config` response made by a System Admin, exposing every operator with disk, backup, DB, or System Admin access. The "secret in plugin config" exposure is known and tracked as Phase 1B; the structural fix replaces the cleartext with a reference. In all cases, a compromised credential grants the access tied to its scope (account-wide for shared-key, namespace-wide for unscoped SAS, role-scoped for SP) until rotated.

Tampering

T-TAMP-1: Unauthorized Post Edit or Delete Propagation

Severity

MEDIUM

Description

Post edits and deletes are signalled inside the `SyncMsg.Post` payload (`EditAt`, `DeleteAt`, `OriginalId`, plus optional `Props.DeleteBy`). The shared-channels framework applies them to the local copy of the corresponding remote post. There is no application-layer verification that the edit or delete came from the original author or that the remote user had edit permissions; the framework trusts that the source Mattermost enforced its own permissions before publishing the sync event.

Code Path

Inbound: `processInboundMessage` → `UnmarshalEnvelope` → `inboundSequencer.Admit` → `p.API.ReceiveSharedChannelSyncMsg` in `server/inbound.go`. The framework looks up the local post via `(RemoteId, source Post.Id)` rather than a plugin-side mapping table.

Mitigation	1. The framework's <code>(RemoteId, source Post.Id)</code> resolution scopes edits to posts that came from the same remote: a sync event on connection A cannot edit a post that originated on connection B, even if its <code>Post.Id</code> field were forged. 2. Connection linkage validation ensures the framework only accepts sync events on channels explicitly linked to the inbound connection. Unlinked targets post an Accept / Block prompt instead of applying content. 3. The source Mattermost enforces its own edit and delete permissions before publishing the sync event. With transport-level authentication scoped per direction, only the legitimate source can publish.
Residual Risk	If transport credentials are compromised (NATS token / password, Azure shared key / SAS, or Service Principal client secret), fabricated edit or delete envelopes can modify previously-relayed posts within the connection's scope. The sequencer's replay protection (T-TAMP-3) limits this to novel forged events, not captured replays.

T-TAMP-2: Reaction and Acknowledgement Tampering

Severity	LOW
Description	Reactions, post acknowledgements, statuses, and membership changes are all carried in the <code>SyncMsg</code> payload. The receiving framework applies them under the source <code>RemoteId</code> . Same author-verification gap as T-TAMP-1: an attacker with transport publish access can fabricate a reaction or acknowledgement attributed to any remote user.
Mitigation	Same as T-TAMP-1, plus: reactions, statuses, and memberships all carry the source-side <code>UserId</code> the framework validates against the upserted remote user pool. A fabricated reaction from a remote user that has never been seen will create that remote user (subject to T-SPOOF-1's Remoteld-isolation guarantee).
Residual Risk	Same as T-TAMP-1.

T-TAMP-3: Replay and Reorder Tampering

Severity	LOW
Description	An attacker with transport publish access could attempt to replay a captured envelope (causing the receiver to re-apply a post / edit / delete) or to reorder envelopes (causing edits to appear before the post they edit).
Code Path	<code>inboundSequencer.Admit</code> in <code>server/inbound_sequencer.go</code> keys on <code>(Epoch, ConnName, ChannelId, Sequence)</code> .

Mitigation	1. Deduplication. Any envelope at or below the per-channel cursor is dropped. A captured envelope replayed verbatim is rejected. 2. Bounded reorder buffer. Envelopes ahead of the cursor sit in an in-memory buffer; gap-fill envelopes promote the cursor and release buffered successors in order. Reordered envelopes are presented to the framework in their original order. 3. Audited gap-deadline. A 1-second ticker drains the sequencer of envelopes whose missing predecessors never arrived. Lost ranges are audited via the <code>InboundSeq*</code> numeric error codes (15300-range) so operators can detect compliance-relevant gaps. 4. Epoch reset. A new <code>Epoch</code> (sender process restart) resets the cursor; the receiver does not conflate sequence numbers across sender generations.
Residual Risk	An attacker who reaches into the transport at-rest store (e.g. via stolen Azure RBAC credentials) and modifies stored envelopes before the receiver consumes them can still substitute content for a sequence slot the receiver has not yet observed. T-TAMP-AZ1 covers the at-rest exposure.

T-TAMP-AZ1: Azure Transport Message Tampering

Severity	MEDIUM
Description	The envelope is not signed or encrypted by the application on any Azure transport: messages in Azure Queue Storage are Base64-encoded, batched WAL blobs in the Azure Blob provider are length-prefixed plaintext envelopes, and Azure Service Bus AMQP message bodies are raw envelope bytes. An attacker with sufficient access to the underlying store (storage account or Service Bus namespace) could modify in-flight or at-rest messages before they are consumed. The Azure Service Bus provider's blob sidecar (file attachments) inherits the same exposure as the Azure Blob provider.
Mitigation	Restrict storage account and Service Bus namespace access via Azure RBAC and network rules. Prefer scoped SAS tokens (Storage) and scoped Service Bus SAS rules over root credentials. Azure Storage and Azure Service Bus both provide transport-level integrity via TLS. Azure Service Bus's PeekLock with <code>DeliveryCount</code> limits how many times a tampered message can be redelivered before the broker auto-moves it to the dead-letter sub-queue.

Repudiation

T-REPUD-1: Audit Trail for Relayed Actions

Severity	MEDIUM
-----------------	---------------

Description	Relayed posts, edits, deletes, and reactions are applied to local state by the shared-channels framework. Forensic reconstruction of cross-domain activity requires correlating evidence across the source Mattermost, the transport, and the destination Mattermost.
Mitigation	Framework-side attribution. Every synced user, post, reaction, and acknowledgement is stamped with the source <code>RemoteId</code> by the receiving framework. The local copy is therefore queryable by source remote; the webapp displays a "Relayed from: alice (via high-side)" indicator on synced users. Operator-facing audit codes. Every non-debug log call in non-test code carries a numeric <code>error_code</code> from <code>server/errcode/codes.go</code> as its first key-value pair. The integer is the stable contract for log grep across plugin versions. Key audit-relevant families: <code>OutboundSyncMsg*</code> / <code>OutboundAttachment*</code> / <code>OutboundProfileImage*</code> / <code>Ping*</code> (10100-range), <code>InboundActive*</code> election events (15200-range), <code>InboundSeq*</code> sequencer events (15300-range), and <code>AzureSPAuditConstructed</code> (12010) for SP credential audit. See Error Codes. Per-envelope ordering. Each sync envelope carries <code>Epoch</code>, <code>Sequence</code>, <code>ConnName</code>, and <code>Timestamp</code>. The receiver's audit codes reference these so a specific local action can be tied back to a specific sender process generation and sequence position. Admin action logs. Team and channel init / teardown actions post bot announcements with the acting user's name. Request-mode approvals and denials are DMed to the requester with the approver's name and the optional reason text.
Residual Risk	End-to-end correlation still requires aligning the source-side framework cursor, the transport's own log (NATS server log, Azure Storage Analytics, Azure Service Bus diagnostic settings), and the destination-side <code>InboundSeq*</code> audit codes. There is no single consolidated audit feed across all three layers.

T-REPUD-AZ1: Limited Azure Audit Trail

Severity	LOW
Description	Azure Storage and Azure Service Bus both provide access logging, but correlating queue, blob, or AMQP operations to specific Cross Guard messages requires additional effort. Azure Queue message content is Base64-encoded in logs; Azure Blob batch blobs are opaque containers from the audit log's perspective; Azure Service Bus diagnostic logs record send/receive operations and message metadata but not application payloads.
Mitigation	Enable Azure Storage Analytics logging on the storage account, and enable Azure Service Bus diagnostic settings for the namespace. Enable Azure Monitor diagnostic

settings on both. Cross Guard logs all inbound/outbound operations with connection names, post IDs, and (for Service Bus) `DeliveryCount` warnings via error code `26004`.

Information Disclosure

T-INFO-1: Message Content Visible to NATS Server

Severity	MEDIUM (in classified environments)
Description	TLS encrypts data in transit between plugin and NATS server, but the NATS server itself processes messages in cleartext. A compromised or malicious NATS server operator can read all relayed message content.
Mitigation	1. Use a dedicated, locally-administered NATS server. Enable TLS with mutual certificate authentication (<code>client_cert</code> / <code>client_key</code> in <code>nats_provider.go</code>). For CDS deployments, the NATS server should be within a controlled enclave. 2. Restrict NATS server network access to only authorized Mattermost servers using IPsec, firewall rules, or network segmentation. Configure NATS ACLs so only authorized Mattermost server clients can subscribe to <code>crossguard.*</code> subjects.
Residual Risk	No application-layer encryption. The NATS server still sees plaintext payloads, but network-level restrictions and ACLs limit exposure to only authorized infrastructure.

T-INFO-2: Team/Channel Topology Leakage

Severity	LOW
Description	Message envelopes contain <code>team_name</code> and <code>channel_name</code> from the source server. An entity with transport read access (NATS subscribe or Azure queue dequeue) learns the team and channel structure of the sending server.
Mitigation	Use dedicated transport channels per connection pair. For NATS, restrict ACLs. For Azure, restrict storage account access via RBAC and network rules.

T-INFO-3: Status Endpoint Data Exposure

Severity	LOW
----------	------------

Description	<code>GET /api/v1/teams/{team_id}/status</code> and <code>GET /api/v1/channels/{channel_id}/status</code> are accessible to any team/channel member (not just admins). These reveal connection names, directions, and remote team name rewrites.
Mitigation	Global status (<code>GET /api/v1/status</code>) is restricted to System Admins. Connection credentials (tokens, passwords, cert paths) are redacted from all status responses.

T-INFO-4: Transport Credentials in Plugin Configuration

Severity	MEDIUM
Description	By default the connections JSON, including NATS tokens, passwords, mTLS key file paths, Azure shared keys, Service Bus SAS connection strings, and Azure AD Service Principal <code>client_secret</code> values, sits as plaintext in <code>config.json</code> on disk and in the Mattermost configuration database. Any operator with disk, backup, or DB access can read it; any System Admin can view it through the System Console and via <code>GET /api/v4/config</code> .
Mitigation	1. Inject the connections JSON via Mattermost's plugin-settings env-var substitution (<code>MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS</code> and <code>...INBOUNDCONNECTIONS</code>). Source the env var from a secret store (Kubernetes Secret, systemd <code>LoadCredential</code>, HashiCorp Vault, Azure Key Vault). With this in place the credential never lands in <code>config.json</code> or in the configuration database. See T-SPOOF-AZ1 and T-INFO-AZ1 for details. 2. For Azure, prefer Service Principal mode over shared-key / SAS so rotation happens in Azure AD without touching plugin config, and so the credential can be scoped via RBAC to a single resource. 3. Connection credentials are redacted from status API responses.
Residual Risk	With env-var injection, the credential is still readable from the Mattermost process environment (root on host, container exec, <code>/proc/<pid>/environ</code> , orchestrator logs that capture env). Without it, every operator with disk, backup, DB, or System Admin access can read the credential.

T-INFO-AZ1: Connection String Exposure in Plugin Config

Severity	MEDIUM
Description	Azure transports store credentials in the plugin's connections JSON: the Azure Storage connection string (or account name plus shared key) for the Azure Queue and Azure Blob providers, and a Service Bus SAS connection string for the Azure Service Bus

provider (with a separate Storage account credential for its blob sidecar). Mattermost does not encrypt plugin settings at rest, so by default these credentials are stored as plaintext in `config.json` on disk and in the Mattermost configuration database. System Admins with System Console access can view them; any operator with disk, backup, or DB access can also read them.

Mitigation

1. Inject the connections JSON via the Mattermost env-var pattern (`MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` and `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_INBOUNDCONNECTIONS`) sourced from a secret store (Kubernetes secrets, systemd `LoadCredential=` , HashiCorp Vault). This keeps the credential out of `config.json` and the Mattermost configuration database. See T-SPOOF-AZ1 for details.
2. Limit System Admin access; the System Console still allows reading of the in-memory plugin settings.
3. Rotate Azure Storage account keys and Service Bus SAS keys on a regular schedule. Update the env-var-managed secret after rotation.
4. Prefer scoped Service Bus SAS rules over `RootManageSharedAccessKey` so an exposed credential grants only the minimum needed rights.

Denial of Service

T-DOS-1: Inbound Message Flood

Severity

MEDIUM

Description

An attacker flooding the transport (NATS publish, Azure queue enqueue, Service Bus send) drives sustained envelope volume into the inbound handler. The inbound handler is deliberately serial under the single-active-receiver lease (no relay semaphore), so the floor on processing throughput is the per-envelope cost of unmarshal + sequencer admission + framework handoff.

Code Path

`processInboundMessage` in `server/inbound.go` runs serially per inbound connection under the lease held in `server/inbound_election.go` . The sequencer in `server/inbound_sequencer.go` applies admission cheaply (a per-channel map lookup); the framework handoff is the bulk of per-envelope work.

Mitigation

1. Transport-level authentication limits who can publish (NATS ACLs, Azure RBAC scoped to the specific resource, Service Bus SAS rules scoped to one queue).
2. Provider-level throttling: Azure Queue Storage and Azure Service Bus both have built-in throughput controls and per-namespace quotas. NATS server max-connections / max-pending settings bound the per-connection cost.

	3. Single-active reception prevents flood amplification across cluster nodes: only one node does inbound work for a given connection, so a flood does not multiply across the cluster. 4. The sequencer's gap-deadline ticker keeps memory bounded even under reorder attack: the in-memory reorder buffer drains after the 1-second deadline regardless of subsequent arrivals.
Residual Risk	A sustained flood from a network-authorized client can still consume the active receiver's CPU and slow legitimate cross-domain sync. There is no per-connection or per-subject rate limit within the plugin itself.

T-DOS-2: Framework Retry Amplification

Severity	LOW
Description	The plugin's outbound hooks do not implement a publish-side retry loop. The shared-channels framework owns retry: a hook return error leaves the framework cursor unadvanced, so the framework retries on its own schedule. If a downstream transport is unreachable for an extended period, the framework will continue calling the hook periodically.
Code Path	The hooks in <code>server/hooks.go</code> return an error on transport failure. The framework re-invokes the hook on its retry schedule (framework-defined, not plugin-configurable).
Mitigation	Context cancellation propagates from the framework into the publish call so graceful shutdown is preserved. <code>OnSharedChannelsPing</code> sees the same unhealthy state via <code>QueueProvider.IsConnected</code> and surfaces it through the framework's per-remote health view.
Residual Risk	Retry rate is framework-controlled and cannot be tuned per-connection from the plugin side. An unreachable transport produces a steady-state retry rate until manually corrected.

T-DOS-3: Single-Active Receiver Lease Bottleneck

Severity	LOW
Description	Because only one cluster node holds the inbound subscription at a time, an attacker could attempt to disrupt the active node specifically (e.g. via a network partition that severs it from the transport but not from Mattermost's KV store). The lease would prevent any other node from taking over for up to 45 seconds (the TTL) plus the time it takes the affected node to release.

Code Path	<code>server/inbound_election.go</code> holds the KV TTL lease with a 45 s TTL and 15 s renewal interval (3x safety ratio).
Mitigation	1. The 3x safety ratio between TTL and renewal interval means a node that loses KV access still releases the lease within one TTL even without a clean shutdown. 2. The active receiver emits <code>InboundActive*</code> audit codes (15200-range) on lease acquisition, renewal, and loss; operators can detect a stuck lease via these. 3. Routine cluster failover (rolling restart, scaling) releases the lease cleanly and triggers immediate takeover.
Residual Risk	A targeted partition that isolates one node from the transport but not from KV produces up to 45 s of inbound stall before failover. This is the trade-off for the no-duplicate-processing guarantee: the alternative (multiple nodes subscribing) would expose duplicate work even in steady state.

T-DOS-AZ1: Azure Transport Saturation

Severity	MEDIUM
Description	An attacker with write access to an Azure transport (Queue, Blob container, or Service Bus queue), or a misconfigured outbound server, could flood the transport with messages, causing the inbound side to process excessive messages and potentially overwhelming the Mattermost server.
Mitigation	Azure Queue Storage and Azure Service Bus both have built-in throttling. The inbound side bounds processing rate per provider: Azure Queue polls at most 32 messages per batch with a 5-second interval, the Azure Blob provider's batch reads are bounded by <code>flush_interval_seconds</code> and per-blob lease, and the Azure Service Bus provider uses PeekLock receive (batch up to 32, 5-second max wait) plus server-side <code>DeliveryCount</code> with automatic dead-letter on <code>MaxDeliveryCount</code> exceedance. The Service Bus DLQ contains poison messages without a custom retry-tracking layer; the plugin logs a WARN with error code <code>26004</code> when <code>DeliveryCount > 1</code> so operators can detect buildup before DLQ moves happen. Monitor queue depth, blob container size, and Service Bus active/DLQ counts via Azure metrics. Set up alerts for unusual growth.

Elevation of Privilege

T-PRIV-1: Connection Management by Non-Admins

Severity	MEDIUM
-----------------	---------------

Description	By default (<code>RestrictToSystemAdmins=false</code>), Team Admins can link / unlink connections at the team level and Channel Admins can manage channel-level connections within an initialized team. A compromised or rogue admin in either tier could enable cross-domain relay on their scope of authority without broader organizational approval.
Code Path	<code>server/command.go</code> 's <code>isTeamAdminOrSystemAdmin</code> and <code>isChannelAdminOrHigher</code> check <code>RestrictToSystemAdmins</code> . <code>server/request.go</code> and <code>server/channel_request.go</code> implement the optional request-mode flows.
Mitigation	1. Lock to System Admins. Enable <code>RestrictToSystemAdmins</code> to gate all link / unlink commands to System Admins. 2. Team-admin request mode. With <code>RestrictToSystemAdmins</code> on and <code>AllowTeamAdminRequests</code> on, a Team Admin running <code>init-team</code> persists a request entry (<code>crossguard-connreq-{teamID}-{connName}</code>) and the plugin DMs every System Admin with an interactive Approve / Deny prompt. <code>teardown-team</code> remains direct (unlinking does not enlarge attack surface). 3. Channel-admin request mode. With <code>RestrictToSystemAdmins</code> on and <code>AllowChannelAdminRequests</code> on, a Channel Admin running <code>init-channel</code> persists a request entry (<code>crossguard-chanconnreq-{channelID}-{connName}</code>) that Team Admins approve via DM prompt (since Team Admins already have channel-link authority in the non-restricted default). <code>teardown-channel</code> remains direct. 4. Inbound acceptance prompts. Even when an outbound connection is linked, the receiving side requires an interactive Accept / Block prompt before applying inbound content. Linking on one side does not grant the other side cross-domain content delivery.
Residual Risk	In the default permissive mode, an admin at the corresponding tier can link a connection within their scope without external approval. The request-mode flows preserve that delegation while requiring two-person consent for linking; unlinking remains delegated to the tier that owns the scope.

T-PRIV-2: Synced Remote User Auto-Membership

Severity	MEDIUM
Description	The shared-channels framework upserts remote users referenced in an inbound <code>SyncMsg</code> and adds them to the destination channel as part of applying the sync event. This bypasses Mattermost's normal invitation / join workflows for the synced channel scope.

Code Path	Inbound: <code>processInboundMessage</code> → <code>p.API.ReceiveSharedChannelSyncMsg</code> . The framework performs user upsert and channel-membership write under the source <code>RemoteId</code> .
Mitigation	1. Connection linkage scope. Auto-membership only happens in channels explicitly linked to an inbound connection by an admin. Unlinked channels post an Accept / Block prompt instead. 2. Remoteld isolation. Synced remote users carry a non-empty <code>RemoteId</code> and the framework refuses to grant them any role on the local server (they inherit <code>system_user</code> only). They cannot become Team Admins or Channel Admins through the sync path. 3. Webapp attribution. The user popover shows "Relayed from: alice (via high-side)" so members of the channel can tell which users are local vs. synced.
Residual Risk	Synced remote users are members of the synced channel by definition; that is the feature. There is no per-user opt-in for the local-side channel; consent is at the channel link level, not at the per-remote-user level.

T-PRIV-3: Active-Receiver Lease Hijack

Severity	LOW
Description	The single-active-receiver lease (<code>crossguard-actinb-{connName}</code>) is held in the Mattermost KV store. An attacker with KV write access (e.g. a compromised plugin running in the same Mattermost cluster) could overwrite the lease to redirect inbound processing to a node they control, then suppress audit events for selected envelopes.
Mitigation	1. KV access requires being a Mattermost plugin running on the cluster, which is a much higher bar than transport-publish access; only plugins installed by the System Admin have any KV access at all. 2. The legitimate active receiver emits a continuous stream of <code>InboundActive*</code> audit codes (15200-range) on each renewal. A hijack interrupts that stream, producing a detectable signature. 3. The sequencer's checkpoint ticker persists cursors to <code>crossguard-seqcur-*</code> ; a hijacker who skips envelopes still leaves a sequence gap that the legitimate-receiver-resumption logs (or the gap-deadline audit codes) will surface.

T-ELEV-AZ1: Blob Container Public Access Misconfiguration

Severity	HIGH
-----------------	------

Description	If an Azure Blob container is accidentally configured with public access (blob or container level), its contents become accessible to anyone on the internet without authentication. This applies to file attachment containers used by the Azure Queue and Azure Service Bus providers and to the combined WAL-plus-files container used by the Azure Blob provider. The Azure Service Bus provider's blob sidecar uses an independent storage account and so must be audited separately from the Service Bus namespace.
Mitigation	The plugin creates containers with private access by default. Enforce "no public access" at the storage account level via Azure policy. Audit container access levels regularly across every storage account in scope (including the independent Service Bus blob sidecar account). Use Azure Defender for Storage to detect public access misconfigurations.

Relay Loop Prevention

Loop prevention is critical for CDS operation. Four independent layers ensure no received content is re-emitted back to its source remote (or to other remotes).

Layer 1: Shared-Channels Framework (intrinsic)

- Every `User`, `Post`, `Reaction`, and acknowledgement received via `ReceiveSharedChannelSyncMsg` is stamped with the source `RemoteId` by the framework.
- The framework's sync cursors key on `(RemoteId, source content ID)`. Content marked as having come from a remote is not eligible to be re-emitted by the framework's outbound queue.
- This is the framework's own loop-prevention guarantee; Cross Guard inherits it by virtue of running on the shared-channels API rather than the lower-level post-lifecycle hooks.

Layer 2: Per-Remote Outbound Suppression

- `server/hooks.go` calls `connNameForRemote(rc.RemoteId)` to resolve the outbound connection for a sync event.
- If the framework asks the plugin to sync content to the same remote it came from, the plugin returns silently instead of publishing. An envelope that arrived on connection A never re-leaves on connection A.

Layer 3: Connection Linkage Validation

- Outbound envelopes are only published to connections explicitly linked to the originating team. An unlinked team produces no outbound traffic.
- Inbound envelopes are only applied if the target team and channel are linked to the receiving inbound connection. Unlinked targets post an Accept / Block prompt and drop the current envelope.

Layer 4: Inbound Deduplication (Sequencer)

- `inboundSequencer.Admit` in `server/inbound_sequencer.go` keys on `(Epoch, ConnName, ChannelId, Sequence)`.
- Any envelope at or below the per-channel cursor is dropped. Combined with the single-active-receiver lease (only one node subscribed per cluster), this absorbs at-least-once redeliveries and replays caused by a sender restart.
- The 1-second gap-deadline ticker audits lost ranges via the `InboundSeq*` numeric error codes (15300-range) so a loop attempt that bypasses Layers 1-3 by injecting forged envelopes is still detectable by operators.

NATS Transport Security

TLS Configuration

- Minimum TLS 1.2 enforced (`tls.VersionTLS12`)
- Optional mutual TLS: client certificate + key pair
- Optional CA certificate for server verification
- Both `client_cert` and `client_key` must be provided together (validated)

Authentication Methods

Method	Configuration	Description
<code>none</code>	No additional fields	No NATS authentication
<code>token</code>	<code>token</code> field	NATS token-based auth

Method	Configuration	Description
credentials	username + password	Username/password auth

Connection Resilience

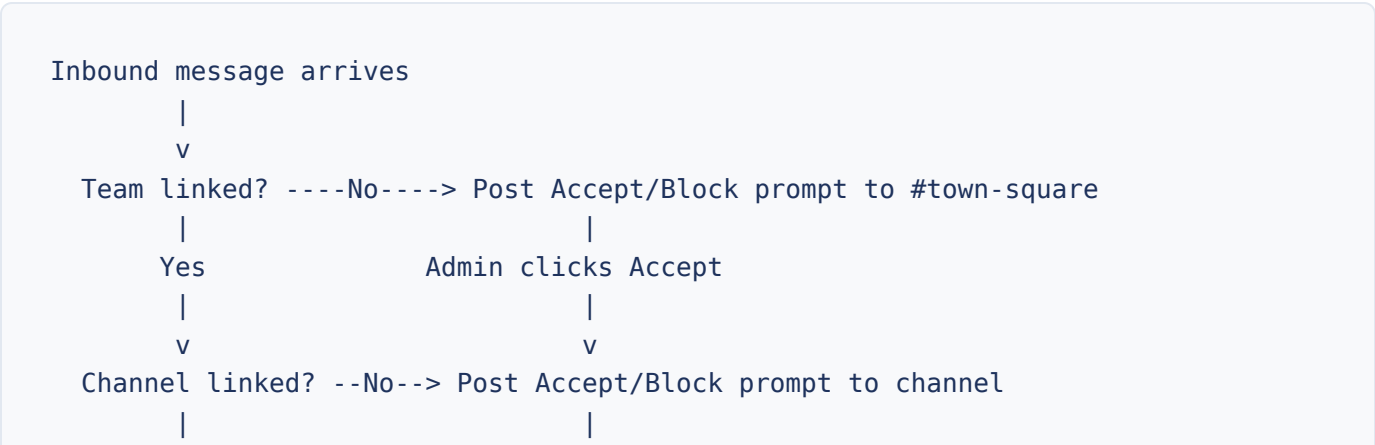
- Persistent connections with unlimited reconnects (`natsMaxReconnects = -1`)
- 2-second reconnect wait
- Disconnect/reconnect event logging

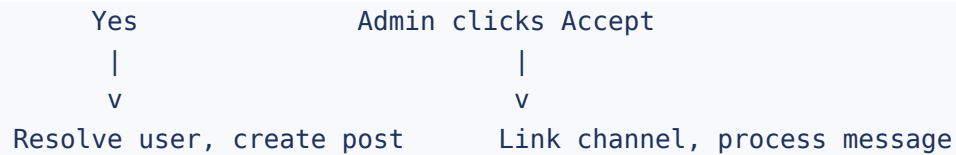
Recommendations by Deployment Sensitivity

Deployment	TLS	Auth	Mutual TLS	Dedicated NATS	NATS ACLs
Development	Optional	Optional	No	No	No
Production (standard)	Required	Token or credentials	Recommended	Recommended	Recommended
CDS (classified)	Required	Credentials	Required	Required	Required

Inbound Connection Acceptance Flow

The prompt-based acceptance workflow prevents unauthorized inbound relay. No messages flow until an admin explicitly accepts the connection at both the team and channel level.





1. Inbound envelope arrives for an unlinked team on an inbound connection.
2. `handleUnlinkedInbound()` in `server/prompt.go` posts an interactive prompt with Accept and Block buttons to `#town-square` on the receiving server. The envelope itself is dropped; subsequent envelopes for the same team follow the prompt state until acted on.
3. A Team Admin or System Admin clicks Accept (POST `/api/v1/prompt/accept`) or Block (POST `/api/v1/prompt/block`). The handler verifies the actor has the corresponding permission.
4. On Accept, the connection is linked to the team and the prompt is replaced with a success message. On Block, the prompt state is recorded as blocked and no further prompts appear until `/crossguard reset-prompt` clears it.
5. The same flow repeats at the channel level via `handleUnlinkedInboundChannel()` and the `/api/v1/prompt/channel/{accept,block}` endpoints. Channel prompts are posted in the target channel rather than town-square.

Locked-Down Mode: Connection Request Flows

In deployments where `RestrictToSystemAdmins` is enabled, the optional `AllowTeamAdminRequests` and `AllowChannelAdminRequests` settings substitute a request-and-approve flow for the direct `init-team` / `init-channel` commands. The accept-flow above for inbound prompts remains unchanged in both modes; request flows cover the outbound link side.

- **Team-admin requests.** A Team Admin running `init-team [connection]` persists a request (`crossguard-connreq-{teamID}-{connName}`) and the plugin DMs every System Admin with an interactive Approve / Deny prompt. Approve calls `POST /api/v1/request/approve`; Deny calls `/deny` which opens an optional-reason dialog, whose submit lands at `/deny-submit`. `teardown-team` remains direct (no approval required to unlink).
- **Channel-admin requests.** A Channel Admin running `init-channel [connection]` persists a request (`crossguard-chanconnreq-{channelID}-{connName}`). The DM prompt goes to **Team Admins** rather than System Admins (Team Admins already have direct channel-link authority in the non-restricted default). The endpoints are `/api/v1/channel-request/{approve,deny,deny-submit}`. `teardown-channel` remains direct.

Deployment Recommendations for CDS Environments

Hardened Configuration Checklist

The following settings are recommended for Cross Guard deployments in classified or sensitive environments.

- 1 Lock connection management to System Admins.** Enable `RestrictToSystemAdmins`. Where the operational model requires delegating linking, enable `AllowTeamAdminRequests` and / or `AllowChannelAdminRequests` for two-person consent: Team Admins submit link requests to System Admins, Channel Admins to Team Admins; unlinking remains direct at each tier (T-PRIV-1).
- 2 Prefer Azure Service Principal authentication on every Azure provider.** Set `auth_mode: "service-principal"` with `tenant_id`, `client_id`, `client_secret` (plus `service_bus_namespace` for Service Bus). Scope the SP to data-plane-only Azure RBAC on the specific queue, container, or namespace. Rotate the secret in Azure AD; pre-provision queues and containers (auto-create is skipped in SP mode). See T-SPOOF-AZ1 and T-INFO-AZ1.
- 3 Keep transport credentials out of `config.json`.** Inject the connections JSON via Mattermost env-var substitution (`MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` and `...INBOUNDCONNECTIONS`) sourced from a secret store. See T-INFO-4 and T-INFO-AZ1.
- 4 Enable TLS and (for NATS) mutual certificate authentication.** For NATS, configure `tls_enabled`, `client_cert`, `client_key`, `ca_cert`. For Azure, TLS is always on at the endpoint.
- 5 Use dedicated transport resources.** Do not share the NATS server, Azure storage account, or Service Bus namespace with other applications. Use separate subjects / queues / namespaces per relay direction.
- 6 Restrict transport network access.** NATS server in a controlled network segment with firewall rules limiting reachability to authorized Mattermost servers; Azure resources behind Virtual Network service endpoints or private endpoints.

- 7 **Restrict transport-level publish / subscribe.** NATS ACLs scoped to `crossguard.*` subjects with per-direction granularity. Azure RBAC roles scoped to the specific resource. Service Bus SAS rules scoped to a single queue rather than namespace-wide root rules.
- 8 **Monitor numeric error codes, not literal log strings.** Operator grep contracts should key on the stable integer codes from `server/errcode/codes.go`. Codes especially relevant to security monitoring: `InboundActive*` (15200-range, election anomalies); `InboundSeq*` (15300-range, replay / reorder / lost-range detection); `AzureSPAuditConstructed` (12010, SP credential rotation audit); `ServiceBusSPCredentialFailed` (26007, SP credential construction failure). See [Error Codes](#).
- 9 **Audit synced remote users periodically.** Enumerate users with a non-empty `RemoteId`. Each carries `UserProps.RemoteUsername` showing the source-side identity. Watch for unexpected remote IDs (T-SPOOF-1).
- 10 **Validate emitted envelopes against the XSD on every release.** Run `make docker-integration-test-validate-wire` in the release pipeline. The mode archives every envelope the running plugin emits and validates each against `schema/crossguard.xsd`, catching any regression that would put non-conformant content on the wire.
- 11 **Enforce `crossguard.` subject prefix (NATS).** All NATS subjects must start with `crossguard.` (enforced at configuration time in `server/configuration.go`). Prevents accidental subscription to unrelated NATS traffic.
- 12 **Document and review team-name rewrite rules.** Rewrites change cross-domain routing; maintain an inventory and review on every change.

Threat Summary Matrix

ID	Threat	Category	Severity	Mitigated?	Control
T-SPOOF-1	Remote user impersonation	Spoofing	MEDIUM	Partial	Framework RemoteId isolation + UserPopover attribution +

ID	Threat	Category	Severity	Mitigated?	Control
					transport-level access control
T-SPOOF-2	Transport message injection	Spoofing	MEDIUM	Partial	TLS + transport auth + sequencer replay protection
T-TAMP-1	Unauthorized edit / delete propagation	Tampering	MEDIUM	Partial	Framework RemoteId scoping + linkage validation
T-TAMP-2	Reaction / acknowledgement tampering	Tampering	LOW	Partial	Same as T-TAMP-1 + remote-user upsert gating
T-TAMP-3	Replay / reorder tampering	Tampering	LOW	Partial	Inbound sequencer Epoch/Sequence dedup + audited gap-deadline
T-REPUD-1	Audit trail for relayed actions	Repudiation	MEDIUM	Partial	Numeric error codes + framework RemoteId + admin action logs
T-INFO-1	Transport sees cleartext envelope	Info Disclosure	MEDIUM	Partial	Dedicated transport + mTLS / RBAC + network restrictions
T-INFO-2	Topology leakage	Info Disclosure	LOW	Partial	Dedicated transport channels + ACLs / RBAC
T-INFO-3	Status endpoint exposure	Info Disclosure	LOW	By design	Credentials redacted; admin-only global status
T-INFO-4	Transport credentials in plugin config	Info Disclosure	MEDIUM	Mitigated	Env-var injection of connections JSON + Azure Service Principal mode

ID	Threat	Category	Severity	Mitigated?	Control
T-DOS-1	Inbound message flood	DoS	MEDIUM	Partial	Transport auth + provider quotas + single-active receiver
T-DOS-2	Framework retry amplification	DoS	LOW	By design	Framework owns retry cadence; ping reflects unhealthy state
T-DOS-3	Single-active lease bottleneck	DoS	LOW	By design	45 s TTL with 3x safety; InboundActive* audit codes detect stalls
T-PRIV-1	Non-admin connection management	Privilege	MEDIUM	Config option	<code>RestrictToSystemAdmins</code> + optional <code>AllowTeamAdminRequests</code> / <code>AllowChannelAdminRequests</code>
T-PRIV-2	Remote-user auto-membership	Privilege	MEDIUM	By design	Linked channels only; framework caps remote users to <code>system_user</code>
T-PRIV-3	Active-receiver lease hijack	Privilege	LOW	Partial	KV-access prerequisite + audit-code stream + sequence cursor checkpoints
T-SPOOF-AZ1	Azure credential compromise (Storage and Service Bus SAS)	Spoofing	HIGH	Partial	Env-var injection + key rotation + Private Endpoints + scoped SAS rules + SAS error sanitization
T-TAMP-	Azure transport message	Tampering	MEDIUM	Partial	Azure RBAC + network rules + TLS

ID	Threat	Category	Severity	Mitigated?	Control
AZ1	tampering (Queue, Blob, Service Bus)				transport integrity + Service Bus DeliveryCount/DLQ
T-REPUD-AZ1	Limited Azure audit trail	Repudiation	LOW	Partial	Azure Storage Analytics + Service Bus diagnostic logs + plugin logs
T-INFO-AZ1	Connection string exposure in config	Info Disclosure	MEDIUM	Partial	Env-var injection + key rotation + scoped SAS rules
T-DOS-AZ1	Azure transport saturation	DoS	MEDIUM	Partial	Azure throttling + bounded inbound batches + Service Bus DeliveryCount/DLQ
T-ELEV-AZ1	Blob container public access (incl. Service Bus blob sidecar)	Privilege	HIGH	By design	Private access default + Azure policy

Azure Transport Security Considerations

When using any Azure transport provider (Azure Queue Storage, Azure Blob Storage, or Azure Service Bus), the following security measures should be implemented:

Common to all Azure transports

- **Private Endpoints:** Configure Azure Private Endpoints for storage accounts and Service Bus namespaces so queue, blob, and AMQP traffic stays within the Azure backbone network.
- **Network Rules:** Use Azure Storage firewall rules and Service Bus IP/VNet filters to restrict access to specific IP ranges or virtual networks.
- **Key Rotation:** Rotate storage account access keys and Service Bus SAS keys on a regular schedule. Update the plugin configuration after rotation.

- **Monitoring:** Enable Azure Monitor diagnostic settings, Storage Analytics, and Service Bus diagnostic logs to track access patterns and detect anomalies.
- **Least Privilege:** For Storage, consider scoped Shared Access Signatures (SAS) over full account-level connection strings. For Service Bus, prefer scoped SAS rules over the namespace-wide `RootManageSharedAccessKey`.

Azure Queue Storage and Azure Blob Storage

- **Auto-provisioned containers/queues:** The plugin auto-creates the configured queue and container with private access if they do not exist. Enforce "no public access" at the storage account level via Azure policy.
- **Single-container footprint (Azure Blob):** The Azure Blob provider stores both batch WAL blobs and deferred file blobs in the same container. Audit access policies and lifecycle rules on that one container.

Azure Service Bus

- **Pre-created queues with broker-side tunables:** The plugin does not auto-create Service Bus queues. Operators provision the queue with a chosen `LockDuration` and `MaxDeliveryCount` in the Azure portal, ARM, or Terraform. These broker-side properties cannot be set from a client; placing them under the same change-control as other Azure infrastructure keeps message-handling SLAs and DLQ policy auditable.
- **SAS connection string handling:** Service Bus authentication is via a SAS connection string of the form `Endpoint=sb://<namespace>.servicebus.windows.net/;SharedAccessKeyName=<rule>;SharedAccessKey=<key>`. The provider strips `SharedAccessKey=`, `SharedAccessSignature=`, and `sig=` query parameters out of any error messages it logs (`sanitizeServiceBusError()`) so SAS material does not leak into operator-visible logs.
- **Dead-letter sub-queue (DLQ):** Service Bus auto-moves messages whose `DeliveryCount` exceeds `MaxDeliveryCount` to the queue's dead-letter sub-queue. The plugin does not inspect or process DLQ messages; operators inspect the DLQ via the Azure portal, `az servicebus queue message`, or Azure monitoring. The plugin emits a WARN with error code `26004` when `DeliveryCount > 1` so poison-message buildup is visible before DLQ moves happen.
- **Independent blob sidecar credentials:** When `file_transfer_enabled` is true, file attachments transit an Azure Blob Storage sidecar that takes its own storage account credentials, separate from the Service Bus namespace. The two stores can live in different

Azure subscriptions and have independent access policies. Audit and rotate the two credentials independently.